ScholarGraph: A LangGraph-Orchestrated Multi-Agent Pipeline for Source-Grounded Literature Review and Heuristic Research-Gap Screening

Harshit Kumar
Dept. of CS&E - AIML, NIE, Mysuru
2023ci_harshitkumar_a@nie.ac.in

Mr. Joyan Prajwal Alvares
Dept. of CS&E, NIE, Mysuru
joyanprajwalalvares@nie.ac.in

Mayank Vyas
Dept. of CS&E, NIE, Mysuru
2023cs_mayankvyas_b@nie.ac.in

Mohammad Abuzaid
Dept. of CS&E, NIE, Mysuru
2023cs_mohammadabuzaid_b@nie.ac.in

Mohammed Farhan Tadpatri
Dept. of CS&E, NIE, Mysuru
2023cs_mohammedfarhantadpatri_b@nie.ac.in

***Abstract*—Literature review involves searching multiple sources, organizing relevant papers, comparing their key characteristics, and identifying areas that may require further investigation. Performing these tasks manually can become time-consuming, especially when the literature spans multiple databases and research themes. In this paper, we present ScholarGraph, a web-based literature review system that organizes this process into six stages using a LangGraph workflow: query planning, multi-source retrieval, structured information extraction, summary generation, graph-based analysis, and report generation. The system retrieves papers from arXiv, Semantic Scholar, and IEEE Xplore, removes duplicate records, and extracts four fields—method, dataset, key metric, and limitation—for comparison across papers. It also constructs a graph of papers, authors, and topic relationships and applies a citation-density heuristic to identify clusters that may represent potential research gaps. The results are presented through a React-based dashboard that includes an overview, comparison matrix, gap-evidence view, report view, and question-answering assistant, with support for PDF and DOCX export. The implementation is supported by unit and integration tests using mocked external services. The current work focuses on the system architecture and implementation; quantitative evaluation of retrieval quality, extraction accuracy, and research-gap validity remains outside the present scope.**

***Index Terms*—literature review automation, multi-agent systems, large language models, citation network, research gap detection, LangGraph**

I. Introduction

Literature review is an essential part of research planning, although the process of conducting it includes several steps. Such steps are multiple database searches, elimination of duplicates, selection of relevant references, extraction of relevant information from each paper, aggregation of results, and discovery of research gaps. Systematic review guidelines in software engineering [1] and in healthcare [2] treat the mentioned steps as a rigorous process. LLMs are capable of writing fluent text reviews; however, they can also generate unsubstantiated facts [3]. Retrieval-Augmented Generation (RAG) conditions a model on retrieved documents in order to enhance the factual support for generated text [4]; moreover, such systems have been extended to accommodate scientific articles [5], [6].

Two challenges remain with respect to automating the entire process. First, when a review is produced from a single prompt, it is not obvious which claims are supported by which sources. Second, references to "research gaps" are usually stated in the form of prose, so that the underlying evidence cannot easily be inspected. Consequently, a system would benefit from (i) the generation of intermediate artifacts that can be inspected, and (ii) robust behavior regardless of the availability of external services or their rate limits.

In the current paper, we introduce ScholarGraph, which addresses these issues through a predetermined six-stage process. Every stage takes a shared state object as input and produces an updated shared state object as output, thus ensuring that intermediate results are stored and accessible through an interface. It fetches papers from multiple bibliographic APIs and extracts four attributes of every paper—method, dataset, key metric, and limitation—for comparison across the retrieved literature. It constructs a heterogeneous graph of papers, authors, and topics, and detects clusters of topics with a below-average citation count, thereby identifying potential gaps along with the relevant subgraph.

As far as the scope of the work goes, it is deliberately modest. The system relies on existing techniques (LLM prompting, API-based data fetching, string-based deduplication, embedding similarity, and graph construction) and does not propose a new learning algorithm. The supplied repository does not contain a quantitative evaluation of extraction accuracy, retrieval recall, or gap validity. Hence, the evidence offered here is limited to the implementation itself, an automated test suite that uses mocked services, and an analysis of its results.

The contribution of this work is the following:

1) A six-stage literature-review pipeline implemented as a LangGraph state machine on a typed shared state, integrating arXiv, Semantic Scholar, and IEEE Xplore.
2) A structured comparison workflow that organizes method, dataset, key metric, and limitation for each

retrieved paper.
3) A mixed graph of papers, authors, citations, similarity relationships, and topic clusters, together with a citation-density heuristic for screening potential research gaps.
4) A provider-agnostic LLM access layer behind a FastAPI service, along with a React interface that supports interactive exploration, question answering, and PDF/DOCX report export.
5) An analysis of the current implementation, its test coverage, and the limitations that need to be addressed before quantitative evaluation.

The remainder of the paper is organized as follows: related work is reviewed in Section II; the problem and requirements are stated in Section III; the methodology is presented in Section IV and the implementation in Section V; the available evidence is described in Section VI and discussed in Section VII; and limitations, future work, and conclusions follow in Sections VIII–X.

II. Related Work

A. Literature Review with Assistance by LLMs

RAG conditions generation on retrieved passages, which helps enhance factual grounding for knowledge-intensive tasks [4]. PaperQA leverages a RAG agent to read the full text of scientific papers and introduces LitQA, a benchmark for extracting information from papers [5]. LitLLM identifies candidate papers through the Semantic Scholar API, ranks them against a user-provided abstract, and constructs a related-work section [6]. Semantic Scholar provides the academic graph and public APIs used by many such applications [7]. ScholarGraph is oriented more towards producing structured per-paper records, a comparison table, and graph-based gap candidates, rather than answering individual questions or drafting a related-work section.

B. LLM Agents and Orchestration Frameworks

ReAct interleaves reasoning and action within a single model loop [8]. Surveys catalog the components used to build autonomous LLM-based agents [9], and AutoGen supports applications built from conversing agents [10]. LangGraph represents an application as a graph in which nodes read and modify shared state [11]. ScholarGraph uses LangGraph but with a fixed linear topology and no conditional routing or cycles; the "agents" in ScholarGraph are therefore role-specialized pipeline stages rather than autonomous planners.

C. Structured Extraction and Hallucination

LLMs can extract structured records from scientific text [12], while research on hallucination highlights the need for care when generated content is used in knowledge-intensive applications [3]. ScholarGraph uses structured extraction to organize information from retrieved papers into a consistent set of fields for comparison and downstream synthesis.

D. Bibliometric Mapping and Gap Discovery

Co-citation analysis captures relatedness between documents defined through shared citing papers [13]. CiteSpace and VOSviewer visualize bibliometric networks to reveal structure and emerging trends [14], [15]. These tools mainly visualize bibliographic relations and do not produce structured per-paper records of method, dataset, and limitation. ScholarGraph builds its graphs using NetworkX [16] and renders them with Cytoscape.js [17]. Its gap-discovery heuristic, which compares the average citation count across topic clusters, is a simple proxy that has not been validated against expert judgment, and no comparison against the tools above was performed.

III. Problem Statement and Requirements

For a given free-text topic q and optional filters ϕ (year range, venue type, keywords), the system should return: (a) a collection P of distinct papers; (b) for each paper $p \in P$, a record r_p containing the methodology, data, metric used, and limitations; (c) a graph $G = (V, E)$ connecting papers, authors, and topics; (d) a collection Γ of gap candidates, each with its evidence subgraph; and (e) a report in both editable and static forms. Existing manual literature review work is time-consuming, and single-prompt LLM workflows are not auditable. The implementation requirements are as follows:

- **R1 (coverage):** query several bibliographic sources to cover different facets of the topic.
- **R2 (consistency):** unify duplicate records that arise from different identifiers.
- **R3 (auditability):** supply an evidence graph for every gap candidate.
- **R4 (resilience):** continue to operate even when the LLM provider, a bibliographic API, or a PDF is unavailable.
- **R5 (deliverables):** provide outputs in PDF and DOCX format and allow follow-up questions over the corpus.

IV. Methodology

A. Pipeline Architecture

The pipeline design is represented by a LangGraph `StateGraph`, with a planner as the source node and edges representing the fixed order of dependencies between pipeline stages (Fig. 1). The pipeline state is a typed dictionary that includes the job identifier, query, filters, sub-queries, papers, extracted fields, summaries, comparison table, graph, gap claims, draft report, and a per-agent status. The per-agent status can be pending, running, done, or error. Table I summarizes the individual pipeline stages.

B. Query Planning

The planner calls the LLM at temperature zero to return a JSON list of two to four sub-queries that view the topic from different perspectives, with Markdown fences stripped before parsing. An empty query raises an error and sets the planner status to error; any other failure, including a malformed response, falls back to a single sub-query equal to the original topic so that the pipeline continues (R5).

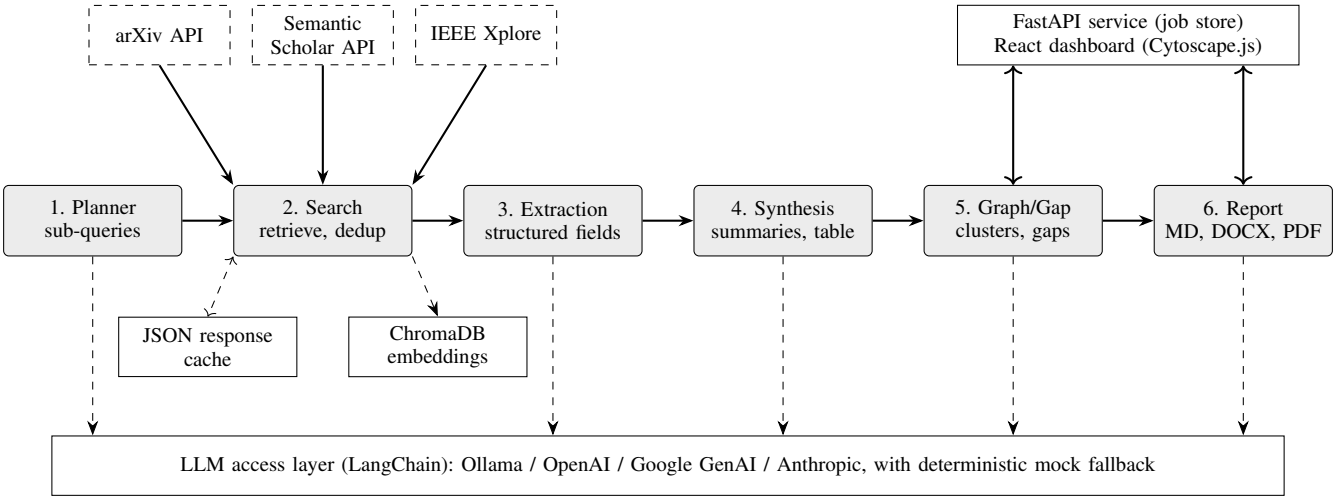


Fig. 1. Architecture of ScholarGraph. Solid arrows denote the fixed stage order and data retrieval; dashed arrows denote auxiliary services. Embeddings written by the Search stage are later read by the Graph/Gap stage and by the question-answering endpoint.

TABLE I
PIPELINE STAGES AND LANGUAGE-MODEL USAGE

Stage	Output	LLM calls
Planner	2–4 sub-queries (fallback: original topic)	1
Search	Deduplicated `PaperMeta` list; embeddings stored	0
Extraction	`FieldRecord` per paper	1 or 2 per paper
Synthesis	Attributed summary and table row per paper	1 per paper
Graph/Gap	Graph, clusters, `GapClaim` list	1 (if $n \geq 15$)
Report	Markdown draft, DOCX, PDF	$2 + g$

C. Multi-Source Retrieval and Deduplication

Retrieval for each sub-query is limited to at most 15 records from each configured source. The arXiv client retrieves paper metadata via a fielded search over titles and abstracts, sorted by relevance within a submission-date range [18]. The Semantic Scholar client retrieves bibliographic data, external identifiers, links to open-access PDFs, and citation counts [19]. The IEEE Xplore client is invoked only when an API key has been configured [20]. Requests are made sequentially, retried with exponential backoff on rate-limit failures, and cached on disk under a hash of the request parameters; a cache lookup checks a local directory and a committed fallback directory in sequence, and a live API call is made only on a cache miss. Raw records are then aggregated into unique papers.

A raw record is treated as equivalent to an existing paper if their DOIs match (case-insensitive), their arXiv identifiers match, or their titles are similar. Titles are similar if the lower-cased alphanumeric strings are identical, or if the Jaccard coefficient of the word sets W_a, W_b of the two titles satisfies

$$J(a,b) = \frac{|W_a \cap W_b|}{|W_a \cup W_b|} \geq 0.8, \tag{1}$$

following the classical definition of the Jaccard coefficient [21]. Since Semantic Scholar expresses references using its own paper identifiers while the corpus relies mainly on the DOI, reference identifiers are translated into corpus identifiers before the graph is built; otherwise, citation edges within the corpus would be silently lost whenever a cited paper is identified by DOI. Additional filters include a year-range filter applied at the source and again on arXiv results, a venue-class heuristic (“arXiv”, “workshop”, or “known non-arXiv venue” for conferences and journals), and a keyword filter requiring at least one keyword in the title or abstract. Finally, the title and abstract of each paper are indexed into a persistent ChromaDB collection.

D. Structured Information Extraction

For each paper whose full-text PDF link is available, the extraction stage attempts to download the document, checks the PDF magic bytes, and extracts text using PyMuPDF [22]. For documents longer than six pages, the system reads the first four and last two pages to keep the input manageable; shorter documents are processed in full. When a full-text PDF is not available, the available bibliographic information and abstract are used instead.

The language model is prompted to extract four fields from each paper: method, dataset, key metric, and limitation. The prompts encourage specific descriptions rather than broad statements and allow the model to use the title and abstract when the available paper information is limited. If any field is left blank or is uninformative, a second inference pass is used to fill the missing information. If an exception occurs or the model response cannot be parsed, a rule-based extractor using keyword lists and regular expressions produces a usable record.

E. Synthesis with Sentence-Level Attribution

During synthesis, the LLM is prompted to produce a three-sentence summary of each paper from its title, abstract, and extracted fields, with every sentence followed by a bracketed

TABLE II
GRAPH SCHEMA CONSTRUCTED BY THE GRAPH/GAP STAGE

Element	Construction rule
Paper node	One per paper; stores year, venue, abstract, citation count
Author node	One per distinct author name string
`AUTHORED_BY`	Paper → each author
`CO_AUTHORED_WITH`	Both directions for each author pair on a paper
`CITES`	Paper → cited paper, only if the cited paper is in the corpus
`SIMILAR_TOPIC`	Both directions if $s_{ij} \geq 0.6$, weighted by s_{ij}
Topic node	One per LLM (or fallback) cluster label
`BELONGS_TO`	Paper → topic, membership score 1.0

source tag such as Abstract, Method, Dataset, Key Metric, or Limitation. The output text is split into sentences, and each sentence is paired with its source tag, or with "General Context" when no tag is present. A row is then created in the comparison table for each paper, combining bibliographic metadata with the four extracted fields: method, dataset, key metric, and limitation.

F. Graph Construction

The graph store builds a NetworkX `MultiDiGraph` [16] following the schema in Table II. Embeddings are retrieved from ChromaDB, and for each pair of papers with embeddings e_i, e_j the cosine similarity

$$s_{ij} = \frac{e_i \cdot e_j}{\|e_i\| \, \|e_j\|} \tag{2}$$

is computed. If $s_{ij} \geq 0.6$, where 0.6 is a fixed constant in the implementation, a bidirectional similarity edge weighted by s_{ij} is added. The default ChromaDB embedding function uses the Sentence Transformers `all-MiniLM-L6-v2` model [23], [24].

G. Cluster-Based Gap Screening

Gap screening is carried out only for corpora with at least 15 papers. The LLM is used to cluster papers into three to five themes, and the response is validated as a non-empty list of objects with a string label, a string description, and a list of paper identifiers; otherwise, a fallback to keyword-based clustering over nine fixed terms is applied. For each cluster c with paper set P_c, restricted to corpus identifiers, the citation density is the mean citation count

$$\delta(c) = \frac{1}{|P_c|} \sum_{p \in P_c} \mathrm{cc}(p), \tag{3}$$

and the set of flagged clusters is

$$\Gamma = \{\, c : \ \delta(c) \leq \mathrm{median}_{c'} \, \delta(c') \,\}. \tag{4}$$

The gap claim for every flagged cluster includes: the label; a description containing the citation density, the median, and the paper identifiers; three suggested directions generated from templates; and a node-link JSON snapshot of the induced subgraph over the cluster's papers, their authors, and the topic node. The suggested directions are string templates parameterized only by the cluster label and are not estimated from the corpus. Because of the construction of Eq. (4), at least half of the clusters are flagged in every run; the mechanism therefore screens candidates rather than testing a hypothesis (Section VIII).

H. Report Generation and Question Answering

In report generation, the LLM is prompted to produce an introduction, a four-part thematic synthesis over all summaries, and a narrative paragraph for every flagged gap. The output is compiled into a Markdown draft, a DOCX file (using python-docx), and a PDF file (using ReportLab), each named by job identifier. The question-answering endpoint retrieves the 15 papers most similar to the question from the vector store, keeps those belonging to the current job, and falls back to the first eight comparison-table rows if none match; the LLM is then prompted with the extracted fields of these papers together with the chat history.

I. Complexity Analysis

Let n be the number of unique papers, N the number of raw records ($N \leq 45k$ for k sub-queries, since at most 15 records are retrieved per sub-query from each of three sources), L the mean title length in words, a_p the number of authors of paper p, d the embedding dimension, and g the number of flagged gaps. Naively deduplicating each raw record against every retained paper costs $O(NnL)$. Building the graph costs $O(\sum_p a_p^2)$ for co-authorship edges and $O(n^2 d)$ for pairwise similarity. Computing cluster statistics costs $O(n)$. For $n \geq 15$, the number of LLM calls satisfies

$$2n + g + 4 \ \leq \ N_{\mathrm{LLM}} \ \leq \ 3n + g + 4, \tag{5}$$

where the upper bound accounts for the optional second extraction pass, so that LLM cost grows linearly with corpus size. However, both the clustering prompt (which contains all titles together with 300-character abstract prefixes) and the thematic-synthesis prompt (which contains all summaries) grow linearly in n with no chunking implemented, so context-window limits bound the usable corpus size.

V. IMPLEMENTATION

A. Software Stack

Table III lists the technologies used in the repository. The backend is implemented as a Python package organized into five modules — `agents`, `clients`, `data`, `orchestration`, and `api` — and its data contracts (`PaperMeta`, `FieldRecord`, `Summary`, `GapClaim`) are described as Pydantic models.

B. Services and Job Lifecycle

The FastAPI service exposes the following endpoints: `POST /query` creates a job identifier and schedules the pipeline as a background task; `GET /status/{id}` reports per-agent status; `GET /results/{id}` returns papers, the comparison

TABLE III
TECHNOLOGY STACK

Layer	Technology
Orchestration	LangGraph `StateGraph` [11]
LLM access	LangChain `init_chat_model`; providers Ollama, OpenAI, Google GenAI, Anthropic
Retrieval	arXiv API, Semantic Scholar Graph API, IEEE Xplore API
PDF text	PyMuPDF
Vector store	ChromaDB persistent client
Graph	NetworkX `MultiDiGraph`
Models	Pydantic
Service	FastAPI, Uvicorn [25]
Reports	ReportLab, python-docx
Frontend	React 19, Vite, Cytoscape.js, lucide-react
Tests	pytest

TABLE IV
SUMMARY OF AUTOMATED TESTS

Module	#	Asserted behavior
Planner	3	At least two string sub-queries; error status for empty or whitespace query
Search	3	Title cleaning; Jaccard threshold behavior; merged output of mocked sources
Extraction	3	Structured field extraction, fallback behavior, and agent flow with mocked PDF and LLM
Synthesis	1	Attribution parsing; comparison-table row contents
Graph/Gap	2	Skip below 15 papers; fallback clustering, graph and subgraph fields
Report	1	Markdown draft; DOCX and PDF files exist
Integration	1	Six stages on 20 mocked papers; all statuses "done"

table, gaps, the graph, summaries, sub-queries, and the report draft; `POST /qa` answers a question; `GET /export/{id}` returns the output as PDF or DOCX; and `GET /health` reports service health. The pipeline runs through a streaming interface, so job status — and therefore agent status — is updated after each stage completes. Jobs are held in an in-memory dictionary, so there is no persistent job store, authentication, or rate limiting, and cross-origin requests are accepted from any origin.

C. LLM Access and Degradation

All model calls pass through a single client that wraps a LangChain chat model built by a factory. The provider is selected from the `LLM_PROVIDER` and `LLM_MODEL` environment variables; if these are unset, the first usable provider is chosen in the order Ollama (if reachable), OpenAI, Google GenAI, and Anthropic, with default model names `llama3.1`, `gpt-4o-mini`, `gemini-2.5-flash`, and `claude-3-5-sonnet-latest` respectively. If no provider is usable, or a call fails or returns an empty response, the client returns a deterministic canned response keyed on the prompt text ("mock mode"). This keeps the pipeline running without credentials and is exploited by the tests, but it also means that a provider failure can produce plausible-looking placeholder content that downstream stages treat as genuine model output (Section VIII).

D. Frontend

The frontend is a single-page React application with five tabs: overview, comparison table, gap evidence, report, and a question-answering assistant. It submits a query with year-range, venue-type, and keyword filters; polls the status endpoint every two seconds; fetches the results on completion; supports searching and sorting in the comparison table; renders node-link data in the graph viewer with Cytoscape.js using the force-directed `cose` layout; persists session state in browser local storage; and targets a hard-coded backend address for local development.

VI. Experimental Setup and Available Evidence

A. Scope

The repository contains no benchmark dataset, gold annotations, training or evaluation scripts, metric logs, or baseline results, so no precision, recall, accuracy, latency, or user-study figures are reported. The available evidence therefore consists of an automated test suite and a committed result artifact, both described below.

B. Automated Tests

The test directory contains 13 unit tests and one integration test. In a fresh execution of the test runner, all 15 collected tests passed — the 14 tests noted above plus one connectivity test run in a scratch directory. Table IV summarizes what each module's tests assert. Because all LLM, bibliographic-API, PDF-download, and PDF-parsing calls are mocked, the tests verify control flow, data contracts, and error handling rather than output quality. The API layer and the frontend have no automated tests.

C. Committed Result Artifact

The repository includes a serialized pipeline result for the query "attention mechanisms", together with cached search responses. The generation script documents that it runs the pipeline using mocked model responses and a cached search-results collection. Table V reports descriptive statistics of this artifact.

VII. Results and Discussion

The tests show that the six stages compose correctly, that each stage populates its output fields under mocked dependencies, and that the fallback paths for small corpora and for clustering execute as intended. They do not show that extraction is accurate, that retrieval is complete, or that the flagged gaps correspond to genuine research opportunities; the bundled artifact should therefore not be read as a performance result.

In particular, 69 of the 73 extraction records share identical values — method "Multi-Head Self-Attention", dataset "Wikitext-103", key metric "Perplexity of 18.2", and one

TABLE V
DESCRIPTIVE STATISTICS OF THE COMMITTED RESULT ARTIFACT

Property	Value
Sub-queries	3
Unique papers	73 (43 arXiv, 30 Semantic Scholar)
Papers with nonzero citation count	24 (maximum 151)
Graph nodes	387 (73 paper, 309 author, 5 topic)
Graph edges	2,527 (2,132 co-authorship, 322 authorship, 73 topic membership)
`CITES` and `SIMILAR_TOPIC` edges	0
Gap claims	3, each with density 0.00 and median 0.00

TABLE VI
IMPLEMENTATION AND VALIDATION STATUS

Capability	Implemented	Tested (mocked)	Empirically validated
Six-stage orchestration	Yes	Yes	No
Multi-source retrieval, dedup	Yes	Partly	No
Structured information extraction	Yes	Yes	No
Graph and gap screening	Yes	Yes	No
Report export (DOCX, PDF)	Yes	Yes	No
Question answering	Yes	No	No
REST API, dashboard	Yes	No	No

shared limitation string — while the remaining four are "Not specified". Similarly, 72 of the 73 summaries are raw JSON strings rather than properly formatted attributed sentences. This uniformity is inconsistent with genuine per-paper extraction and is instead consistent with output from the mock client. Likewise, the cluster labels ("Attention", "Efficiency", "General Research", "Transformer", "Llm") and their descriptions match the keyword-fallback clustering rather than an LLM-produced clustering. The artifact also lacks `CITES` and `SIMILAR_TOPIC` edges and lacks the URL field used by the current data model. These observations suggest that the artifact should be treated as a demonstration of pipeline and fallback behavior rather than as evidence of extraction quality.

Two observations from the artifact remain informative about the design. First, the repeated extraction values show how deterministic mock responses can make different papers appear artificially similar when the pipeline is executed without a live language-model provider. Second, when the median cluster density is zero, the heuristic can flag several low-density clusters, including clusters containing very few papers. This illustrates the sensitivity of the current gap-screening mechanism to missing or low citation counts.

Table VI separates what is implemented, what is exercised by tests, and what remains to be empirically validated.

A. Proposed Evaluation Protocol (Not Yet Performed)

A meaningful evaluation would require, at minimum: (i) manual annotation of a sample of papers to measure field-level precision and recall of extraction; (ii) retrieval recall measured against a reference list drawn from an existing systematic review; (iii) an expert assessment of whether flagged gaps are plausible and non-trivial, compared against a random-cluster baseline; and (iv) a comparison of per-paper factual error rates against a single-prompt baseline. None of these has yet been carried out.

VIII. LIMITATIONS

- **No quantitative validation.** Extraction accuracy, retrieval coverage, and gap validity are unmeasured.
- **Extraction reliability.** The extracted method, dataset, key metric, and limitation depend on the quality of the available paper text and the language-model response. When information is incomplete, the system may infer values or fall back to rule-based extraction, so the resulting comparison fields should not be treated as authoritative without checking the original papers.
- **Silent mock fallback.** A failed LLM call returns canned text without raising an error, so the resulting records can be indistinguishable from real model output in the summaries and reports.
- **Gap heuristic.** Mean citation count depends on paper age and data source; arXiv records typically carry a citation count of zero, and by construction Eq. (4) flags at least half of all clusters in every run. A code comment refers to a recent-years window, but the density is actually computed over total citation counts, and the suggested directions are template text rather than corpus-derived recommendations.
- **Embedding fallback.** If the default embedding model cannot be loaded, the vector store falls back to hash-seeded pseudo-random vectors that carry no semantic information, so similarity edges and question-answering retrieval are not meaningful in that mode.
- **Retrieval and scale.** Sources are queried sequentially with a fixed per-source limit; venue-type filtering is heuristic; missing years default to 2000; prompts grow linearly with corpus size with no chunking; and only the first four and last two pages of a PDF are read.
- **Question answering.** Context consists of extracted fields only, not abstracts or passages, and the vector collection is shared across jobs and filtered to the current job only after retrieval, which can discard relevant papers.
- **Security and deployment.** Jobs are held only in memory, the service has no authentication, and cross-origin access is unrestricted. The environment template enables LangChain tracing flags, which would transmit prompt text to a third-party service if credentials were supplied, and the frontend targets a fixed local address.

IX. Future Work

Realistic extensions include carrying out the evaluation protocol described in Section VII; improving extraction through passage-level retrieval and stronger context selection; defining a gap criterion that normalizes citation counts by paper age and source and is compared against baselines; introducing conditional routing or retries into the LangGraph topology; issuing source queries concurrently; adding a persistent job store with authentication; and adding automated tests for the API layer and the frontend. None of these is implemented in the current system.

X. Conclusion

ScholarGraph is an implemented system that integrates multi-source retrieval, LLM-based structured extraction, graph construction, and cluster-level gap screening within a LangGraph pipeline, and exposes the results through a web service and a dashboard. Its contribution lies in bringing these components together into a single workflow that supports paper comparison, graph-based exploration, report generation, and question answering. The automated tests support the structural operation of the pipeline under mocked dependencies, but the bundled result artifact originates from mocked model output and does not support any claim about extraction quality. The accuracy of the extracted fields, the completeness of retrieval, and the validity of the flagged gaps all remain to be evaluated.

References

[1] B. Kitchenham and S. Charters, "Guidelines for performing systematic literature reviews in software engineering," Keele Univ. and Univ. Durham, Tech. Rep. EBSE-2007-01, 2007.

[2] M. J. Page *et al.*, "The PRISMA 2020 statement: An updated guideline for reporting systematic reviews," *BMJ*, vol. 372, Art. no. n71, 2021.

[3] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, Art. no. 248, 2023.

[4] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[5] J. Lála *et al.*, "PaperQA: Retrieval-augmented generative agent for scientific research," arXiv:2312.07559, 2023.

[6] S. Agarwal *et al.*, "LitLLM: A toolkit for scientific literature review," arXiv:2402.01788, 2024.

[7] R. Kinney *et al.*, "The Semantic Scholar Open Data Platform," arXiv:2301.10140, 2023.

[8] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[9] L. Wang *et al.*, "A survey on large language model based autonomous agents," *Front. Comput. Sci.*, vol. 18, no. 6, Art. no. 186345, 2024.

[10] Q. Wu *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," arXiv:2308.08155, 2023.

[11] LangChain, Inc., "LangGraph." [Online]. Available: https://github.com/langchain-ai/langgraph (accessed Sep. 24, 2026).

[12] J. Dagdelen *et al.*, "Structured information extraction from scientific text with large language models," *Nat. Commun.*, vol. 15, Art. no. 1418, 2024.

[13] H. Small, "Co-citation in the scientific literature: A new measure of the relationship between two documents," *J. Amer. Soc. Inf. Sci.*, vol. 24, no. 4, pp. 265–269, 1973.

[14] C. Chen, "CiteSpace II: Detecting and visualizing emerging trends and transient patterns in scientific literature," *J. Amer. Soc. Inf. Sci. Technol.*, vol. 57, no. 3, pp. 359–377, 2006.

[15] N. J. van Eck and L. Waltman, "Software survey: VOSviewer, a computer program for bibliometric mapping," *Scientometrics*, vol. 84, no. 2, pp. 523–538, 2010.

[16] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX," in *Proc. 7th Python Sci. Conf. (SciPy)*, 2008, pp. 11–15.

[17] M. Franz *et al.*, "Cytoscape.js: A graph theory library for visualisation and analysis," *Bioinformatics*, vol. 32, no. 2, pp. 309–311, 2016.

[18] arXiv, "arXiv API user's manual." [Online]. Available: https://info.arxiv.org/help/api/user-manual.html (accessed Sep. 24, 2026).

[19] Allen Institute for AI, "Semantic Scholar Academic Graph API." [Online]. Available: https://api.semanticscholar.org/api-docs/ (accessed Sep. 24, 2026).

[20] IEEE, "IEEE Xplore API portal." [Online]. Available: https://developer.ieee.org/ (accessed Sep. 24, 2026).

[21] P. Jaccard, "The distribution of the flora in the alpine zone," *New Phytol.*, vol. 11, no. 2, pp. 37–50, 1912.

[22] Artifex Software, "PyMuPDF documentation." [Online]. Available: https://pymupdf.readthedocs.io/ (accessed Sep. 24, 2026).

[23] Chroma, "Embedding functions." [Online]. Available: https://docs.trychroma.com/docs/embeddings/embedding-functions (accessed Sep. 24, 2026).

[24] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. Empirical Methods Natural Lang. Process. and 9th Int. Joint Conf. Natural Lang. Process. (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.

[25] S. Ramírez, "FastAPI." [Online]. Available: https://fastapi.tiangolo.com/ (accessed Sep. 24, 2026).